Federated Multimodal Graph Learning Framework

Mortality & Sepsis Prediction using the eICU Dataset

Progress Report — Steps 1 through 4

Date: October 7, 2026
Dataset: eICU Collaborative Research Database

Table of Contents

1. Executive Summary
2. Dataset Overview
3. Step 1 — Data Preparation
4. Step 2 — Modality-Specific Feature Learning
   4.1 Tabular Encoder (MLP)
   4.2 Time-Series Encoder (LSTM)
   4.3 Text Encoder (MiniBERT)
   4.4 Federated Pre-training
5. Step 3 — Multimodal Fusion
6. Step 4 — Intra-Hospital Patient Graph Construction
7. Interactive Visualization Playground
8. Codebase Structure
9. Summary & Next Steps

1. Executive Summary

This report documents the implementation progress of the first four steps of our Federated Multimodal Graph Learning Framework for predicting in-hospital mortality and sepsis. The framework treats each hospital as an independent federated learning client, ensuring that raw patient data never leaves the hospital boundary.

We have successfully completed:

- Step 1 — Data Preparation (partitioning the eICU dataset by hospital)
- Step 2 — Modality-Specific Feature Learning (Tabular MLP, Time-Series LSTM, and Text MiniBERT encoders)
- Step 3 — Multimodal Fusion (concatenation + projection network)
- Step 4 — Intra-Hospital Patient Graph Construction (k-NN and threshold-based graphs)

Additionally, a federated pre-training script for the Tabular Autoencoder using FedAvg has been implemented and executed, and a React-based interactive visualization playground has been built to inspect the patient graphs.

2. Dataset Overview

Metric	Value
Source	eICU Collaborative Research Database
Total Hospitals	208
Total Patients	200,859
Tabular Features per Patient	72 (demographics, vitals, diagnoses, medications)
Time-Series Features per Timestep	51 (labs, infusions, treatments)
Time-Series Resolution	Hourly bins (padded/truncated to 48 hours)
Text Features	Lab notes, infusion names, medication names
Prediction Targets	Hospital mortality, ICU mortality

Data is partitioned by hospitalid so that each hospital directory contains independent tabular and timeseries parquet files. Each hospital performs all preprocessing locally, ensuring no cross-hospital data sharing occurs at the data level.

3. Step 1 — Data Preparation

Module: src/datasets.py

Classes: EICUTabularDataset, EICUTimeSeriesDataset

What It Does

1. Loads the .parquet files for a single hospital.

2. Separates input features from target labels (mortality flags).
3. Applies z-score normalization using globally computed statistics.
4. Pads/truncates time-series sequences to a fixed window of 48 hours.
5. Extracts unstructured text from free-text columns (lab_other_names, infusion_other_names, med_other_names).

Sample Output (Hospital 102)

Property	Value
Patients	31
Tabular input dimension	67 features
Time-series input dimension	46 features × 48 timesteps
Mortality cases	3 out of 31 (9.7%)

4. Step 2 — Modality-Specific Feature Learning

Three separate neural network encoders process each modality independently, each producing a 64-dimensional patient embedding.

4.1 Tabular Encoder (MLP)

Property	Value
Architecture	3-layer MLP (input → 128 → 64 → 64)
Normalization	BatchNorm after each hidden layer
Activation	ReLU
Regularization	Dropout (p=0.3)
Input	67 static clinical features
Output	64D embedding per patient

4.2 Time-Series Encoder (LSTM)

Property	Value
Architecture	2-layer Stacked LSTM + Linear Projection
Hidden dimension	64
Sequence handling	pack_padded_sequence for variable-length inputs
Output extraction	Last valid hidden state
Input	(48, 46) — 48 hours × 46 features
Output	64D embedding per patient

4.3 Text Encoder (MiniBERT)

Property	Value

Base model	prajjwal1/bert-mini (HuggingFace)
Parameters	~11.3M (pre-trained)
Tokenizer	BertTokenizer
Max sequence length	512 tokens
Projection	Linear(256→128) → BN → ReLU → Linear(128→64)
Input	Concatenated clinical text strings
Output	64D embedding per patient

Smoke Test Results (Hospital 102)

```
[PASS] All smoke tests passed!
   Tabular  : (67,) → (64,)
   LSTM     : (48, 46) → (64,)
   Text     : (variable length str) → (64,)
```

4.4 Federated Pre-training (Tabular Autoencoder)

Parameter	Value
Communication rounds	15
Local epochs per round	3
Client fraction per round	15% (~31 hospitals)
Batch size	32
Learning rate	1e-3
Aggregation strategy	FedAvg (weighted by patient count)

The resulting pre-trained weights are saved at pretrained_weights/global_tabular_ae.pth.

5. Step 3 — Multimodal Fusion

Module: src/fusion.py

Class: MultimodalFusion

The three 64D modality embeddings are concatenated to form a 192D vector, which is then projected through a smoothing MLP:

```
Tabular Emb (64) ─┐
                  ├─ Concat (192) → Linear → BN → ReLU → Dropout
TimeSeries Emb(64)─┤                → Linear(192→128) → BN → ReLU
Text Emb (64) ────┘
                    Output: Unified Patient Embedding (128D)
```

Property	Value

Input	3 × 64D = 192D concatenated embedding
Hidden layer	192 → 192 (with BatchNorm, ReLU, Dropout)
Output	128D unified patient embedding

6. Step 4 — Intra-Hospital Patient Graph Construction

Module: src/graph_builder.py

Functions: build_knn_graph, build_threshold_graph

6.1 k-Nearest Neighbors (k-NN) Graph

- Computes pairwise cosine similarity between all patient embeddings.
- Connects each patient to their top-k most similar neighbors.
- Produces directed edges (source → target) with similarity scores as edge weights.

6.2 Threshold-Based Graph

- Computes pairwise cosine similarity.
- Creates an edge between patients whose similarity exceeds a threshold (e.g., 0.75).
- Produces undirected edges with similarity scores as weights.

Privacy Enforcement

Since each batch only contains patients from one hospital (by construction of the federated data partitioning), cross-hospital edges are mathematically impossible. No additional filtering is required.

Graph Output (Hospital 102, k=3 KNN)

Property	Value
Nodes (patients)	31
Edges	49
Edge format	PyTorch Geometric edge_index (2 × E)
Edge weights	Cosine similarity scores
Survived nodes	28 (green)
High-risk nodes	3 (red)

7. Interactive Visualization Playground

To facilitate inspection and demonstration, we built a React + Vite frontend that renders the patient graph as an interactive force-directed layout.

- Force-directed graph with physics simulation (nodes repel, edges attract)
- Color-coded nodes — Green (Stable/Survived), Red (High Risk/Expired)

- Click-to-inspect — Selecting a patient reveals clinical text and neighbor count
- Real data — Generated from actual hospital_102 patient data through the full pipeline



Figure 1: Patient Graph Overview — Hospital 102

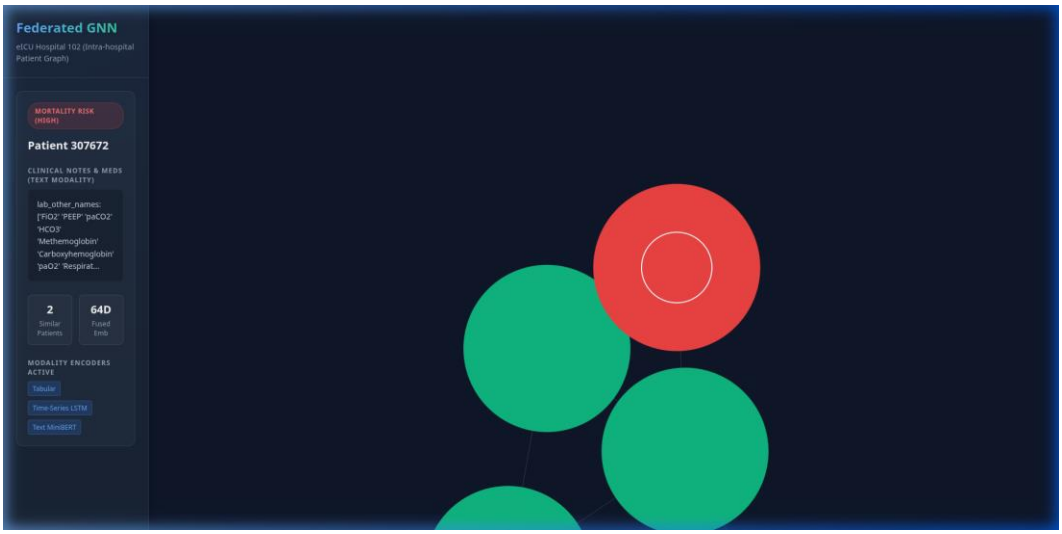


Figure 2: Selected Patient Details — Clinical Text and Modality Tags

8. Codebase Structure

```
project-root/
├── processed/                    # Raw data (208 hospitals)
│   ├── hospitals/hospital_*/
│   ├── feature_schema.json
│   └── timeseries_stats.json
├── src/                          # Core modules
```

```
│   ├── datasets.py              # Step 1
│   ├── encoders.py              # Step 2
│   ├── fusion.py                # Step 3
│   └── graph_builder.py         # Step 4
├── scripts/                     # Executable scripts
├── tests/                       # Smoke tests
├── pretrained_weights/          # Saved model weights
└── frontend-playground/         # React visualization
```

9. Summary & Next Steps

Completed (Steps 1–4)

Step	Component	Status
1	Data Preparation	✅Complete
2	Tabular Encoder (MLP)	✅Complete
2	Time-Series Encoder (LSTM)	✅Complete
2	Text Encoder (MiniBERT)	✅Complete
2	Federated Pre-training (AE + FedAvg)	✅Complete
3	Multimodal Fusion	✅Complete
4	Patient Graph Construction	✅Complete

Remaining (Steps 5–8)

Step	Component	Status
5	Graph Neural Network (GraphSAGE / GAT)	☐Not started
6	Multi-task Prediction Heads	☐Not started
7	Federated Client Training Loop	☐Not started
8	Central Server Aggregation (FedAvg)	☐Not started

Planned Next Actions

6. Implement a Graph Neural Network module (GraphSAGE or GAT) for graph-enhanced patient embeddings.
7. Add dual prediction heads for mortality and sepsis classification with a multi-task loss function.
8. Build the full federated training loop coordinating all 208 hospital clients.
9. Evaluate using AUROC, AUPRC, Accuracy, Precision, Recall, and F1-score.